Live session Content

Github repo link: https://github.com/sureshbabuvikram/FSD-B6-DayWiseCode
MdnLink:https://developer.mozilla.org/en-US/docs/Web/HTML

Day1 :

- What is a full-stack web development
- Introduction to Web browser JavaScript V8
- How the Server looks at the URL
- Browser js vs Node js
- What is Frontend, Backend & Databases
- Evolution of HTTP, HTTP Methods
- Request & Response cycle

1. What is Full Stack Web Development?

Full Stack Web Development refers to the development of both the front-end (client-side) and back-end (server-side) portions of a web application. A full-stack developer is skilled in various technologies that cover both these aspects and can work on all layers of the application stack, including the database.

- **Frontend:** Involves the visual part of the web application that users interact with (HTML, CSS, JavaScript).
- **Backend:** Manages data and business logic (Node.js, Express.js, Python, Java).
- **Database:** Stores and retrieves data (MongoDB, MySQL).

Example:

- A full stack developer might build a web page (frontend) that lets users submit a form, write the server logic (backend) to process that form and store the submitted data in a database.

---

2. Introduction to Web Browser JavaScript V8

Content: JavaScript V8 is an open-source JavaScript engine developed by Google, which is used in the Chrome browser and Node.js. V8 compiles JavaScript directly to native machine code, which makes it very fast.

- **JavaScript Engine:** A program or interpreter that executes JavaScript code. V8 is one of the most popular engines.
- **Performance:** V8 improves performance by compiling JavaScript into machine code before executing it.

Example: When you write JavaScript code on a web page (e.g., `alert('Hello World!');`), the V8 engine inside the Chrome browser compiles and runs this code, displaying the alert message to the user.

---

3. How the Server Looks at the URL

When a user enters a URL in the browser, the browser sends an HTTP request to the server. The server processes the request and returns a response, which is then rendered by the browser.

- **URL (Uniform Resource Locator):** Specifies the address of a web resource on the internet.
- **HTTP Request:** Sent by the browser, it includes the URL, method (GET, POST), headers, and sometimes body data.
- **Server Response:** The server returns the requested resource (like an HTML page) or an error message (like 404 Not Found).

Example:

- URL: `http://example.com/about`
- The browser sends a GET request to the server.
- The server looks at the `/about` route and returns the corresponding HTML page.

---

4. Browser JavaScript vs Node.js

JavaScript can run in both the browser and on a server (using Node.js). While the core language is the same, there are differences in the environment.

- **Browser JavaScript:**
  - Runs in the browser.
  - Has access to the DOM (Document Object Model) for manipulating HTML and CSS.
  - Used for client-side interactivity.
- **Node.js:**
  - Runs on the server.
  - Has access to file systems, databases, and network operations.
  - Used for server-side operations like handling HTTP requests.

Example:

- **Browser JavaScript:** `document.querySelector('h1').innerText = 'Hello, World!';`
  - Changes the text inside the first <h1> tag to "Hello, World!".
- **Node.js:** `fs.readFile('example.txt', 'utf8', (err, data) => { console.log(data); });`
  - Reads the content of a file and logs it to the console.

---

5. What is Frontend, Backend & Databases

These are the three main components of a web application:

- **Frontend:**
  - The visual part of the website that users interact with.
  - Technologies: HTML, CSS, JavaScript.
  - Example: The design, buttons, and forms you see on a website.
- **Backend:**
  - The server-side logic that processes requests from the frontend.
  - Technologies: Node.js, Express.js, Python, Java.
  - Example: Handling form submissions, authentication, and business logic.
- **Database:**
  - Stores and retrieves data.
  - Technologies: MongoDB, MySQL, PostgreSQL.
  - Example: A database stores user information, such as login credentials.

Example: When a user logs into a website:

- **Frontend:** The login form.

- **Backend:** The server code that verifies the username and password.
- **Database:** Where the user's credentials are stored.

---

6. Evolution of HTTP & HTTP Methods

HTTP (HyperText Transfer Protocol) is the protocol used for transferring web pages on the internet. It has evolved over time to become more secure and efficient.

- **HTTP 1.0/1.1:** Early versions with basic features like GET and POST methods.
- **HTTP/2:** Introduced multiplexing, allowing multiple requests to be sent over a single connection.
- **HTTP/3:** Uses QUIC protocol for faster, more reliable connections.

HTTP Methods:

- **GET:** Retrieve data from the server.
- **POST:** Send data to the server to create/update resources.
- **PUT:** Update a resource on the server.
- **DELETE:** Remove a resource from the server.

Example:

- **GET /articles:** Fetch a list of articles.
- **POST /articles:** Create a new article.

---

7. Request & Response Cycle

The request-response cycle is the sequence of events that takes place when a client (browser) interacts with a server.

1. **Client Request:**
    - The client sends an HTTP request to the server.
    - Includes a method (GET, POST), URL, headers, and body data.
2. **Server Processing:**
    - The server processes the request, which might involve querying a database or executing some logic.
3. **Server Response:**
    - The server sends back an HTTP response with a status code (e.g., 200 OK, 404 Not Found), headers, and body content.

4. **Client Rendering:**
    - The browser renders the response content, such as HTML, CSS, and JavaScript, to display the web page.

Example:

- **Request:** The browser requests `http://example.com/home`.
- **Response:** The server responds with the HTML content of the home page, which the browser then renders.

Note: Visual Studio Code install - For the next session

Day2 :

- Alerts & Prompts
- Operators - Arithmetic & Logical
- Variables & Data Types

1. Alerts & Prompts

Alerts and **Prompts** are methods provided by the browser to interact with users by displaying messages or getting input. They are part of the Window object in JavaScript.

Alerts

- **`alert()`**: Displays a message to the user in a dialog box with an OK button.
- **Use Case**: Alerting users about important information or warnings.

Example:

```
alert("Welcome to the website!"); // real-time example validation alert
```

Prompts

- **`prompt()`**: Displays a dialog box that asks the user for input. It contains a text field and OK/Cancel buttons.
- **Use Case**: Gathering user input directly from the browser.

Example:

```
let name = prompt("What is your name?");
alert("Hello, " + name + "!");
```

---

2. Operators - Arithmetic & Logical

Arithmetic Operators

Arithmetic operators perform mathematical calculations on numbers.

- **`+` (Addition)**: Adds two numbers.
- **`-` (Subtraction)**: Subtracts the second number from the first.
- **`*` (Multiplication)**: Multiplies two numbers.
- **`/` (Division)**: Divides the first number by the second.
- **`%` (Modulus)**: Returns the remainder of a division.
- **`**` (Exponentiation)**: Raises the first number to the power of the second.

Examples:

```
let x = 10;
let y = 3;

console.log(x + y); // Output: 13
console.log(x - y); // Output: 7
console.log(x * y); // Output: 30
console.log(x / y); // Output: 3.3333...
console.log(x % y); // Output: 1
console.log(x ** y); // Output: 1000 (10^3)
```

Logical Operators

Logical operators are used to perform logical operations, usually in conditions (like `if` statements).

- **`&&` (Logical AND)**: Returns `true` if both operands are true.
- **`||` (Logical OR)**: Returns `true` if at least one of the operands is true.
- **`!` (Logical NOT)**: Inverts the value of a boolean expression.

Examples:

```
let a = true;
let b = false;

console.log(a && b); // Output: false (both must be true)
console.log(a || b); // Output: true (at least one is true)
console.log(!a);     // Output: false (inverts true to false)
```

---

3. Variables & Data Types a = 10 , name= “suresh”, v=10.10 log(name)

Variables

Variables in JavaScript are used to store data(any type) that can be referenced and manipulated later in the code.

- **`var`**: Declares a variable (function-scoped / Global Scope).
- **`let`**: Declares a block-scoped variable.(local)
- **`const`**: Declares a block-scoped constant (cannot be reassigned).

Example:

```
var name = "Suresh";
let age = 35;
const country = "Tamilnadu";

console.log(name);
console.log(age);
console.log(country);
```

What is ES in JavaScript?

ES stands for **ECMAScript**, which is the standardized scripting language that JavaScript is based on. ECMAScript is maintained by ECMA International through the TC39 committee, and it provides the specifications that JavaScript implementations follow.

Difference between Let and Var:

Scope: var is function-scoped or globally scoped, while let is block-scoped.
Hoisting: var variables are hoisted and initialized with undefined, whereas let variables are hoisted but not initialized.
Re-declaration: var allows re-declaration, but let does not allow re-declaration within the same scope.

Data Types

JavaScript has several basic data types, including:

String: Text data, enclosed in quotes.

```
let greeting = "Hello, World!";
```

Number: Numeric data, including integers and floating-point numbers.

```
let score = 95;
let temperature = 36.6;
```

Boolean: Logical data, either `true` or `false`.

```
let isLoggedIn = true;
```

Null: A special value representing "no value."

```
let empty = null;
```

Undefined: A variable that has been declared but not assigned a value.

```
let notAssigned;
```

Object: A complex data type used to store collections of data.

```
let person = {
  name: "Alice",
  age: 25,
  isStudent: false
};
```

Array: A special type of object used to store lists of values.

```
let numbers = [1, 2, 3, 4, 5];
```

Day3 :

1. What is a Function?

A function in JavaScript is a reusable block of code designed to perform a specific task. Functions help you structure your code, making it modular, maintainable, and reusable.

2. Declaring and Calling a Function

Function Declaration: A function is declared using the `function` keyword, followed by the function name, parentheses `()`, and a code block `{}` containing the statements.

Syntax:

```
function functionName(parameters) {
  // Code to be executed
}
```

Example:

```
function display(name) {
  console.log("Hello, " + name + "!");
}
```

```
display("suresh");
```

3. Types of Functions in JavaScript

JavaScript supports various types of functions, each serving different purposes. Here are the main types:

a. Function Declaration // Normal Function

A function declared with the `function` keyword.

Example:

```
function add(a, b) {
  return a + b;
}

console.log(add(5, 3)); // Output: 8
```

b. Function Expression // Anonymous Function

In a function expression, a function is assigned to a variable. This allows functions to be anonymous (without a name).

Example:

```
const subtract = function(a, b) {
  return a - b;
};

console.log(subtract(5, 3)); // Output: 2
```

c. Arrow Function (ES6)

Introduced in ES6, arrow functions provide a shorter syntax and are often used for writing small functions.

Syntax:

```
const functionName = (parameters) => {
  // Code to be executed
};
```

Example:

```
const multiply = (a, b) => a * b;

console.log(multiply(5, 3)); // Output: 15
```

Note: Arrow functions have a concise syntax. The `return` keyword can be omitted if the function body consists of a single expression.

d. Immediately Invoked Function Expression (IIFE)

An IIFE is a function that is executed immediately after it is defined. It is often used to avoid polluting the global scope.

Example:

```
(function() {
  console.log("This function runs immediately!");
})();

let a=10;
(function(){
    let a=20
    console.log(a);
})();
```

e. Anonymous Function

An anonymous function is a function without a name. It is often used in situations where the function does not need to be reused.

Example:

```
setTimeout(function() {
```

```
  console.log("This runs after 2 seconds");
}, 2000);
```

f. Higher-Order Function

A higher-order function is a function that takes another function as an argument or returns a function as a result.

Example:

```
function display(name) {
  return function(msg) {
    console.log(msg + ", " + name);
  };
}

const retFun= display("suresh");
retFun("Hello"); // Output: "Hello, suresh"
```

g. Callback Function

A callback function is a function passed as an argument to another function and executed after the completion of that function.

Example:

```
function processUserInput(callback) {
  const name = "Suresh";
  callback(name);
}

processUserInput(function(name) {
  console.log("Hello, " + name);
});
// Output: "Hello, Suresh"
```

4. Returning Values from Functions

Functions can return a value using the `return` statement. Once the `return` statement is executed, the function exits, and no further code in the function is executed.

Example:

```
function square(num) {
  return num * num;
}

console.log(square(4)); // Output: 16
```

If-Else statements in JavaScript are used to execute different blocks of code based on different conditions. They are fundamental to controlling the flow of a program.

1. Basic If-Else Statement

Explanation:

- The if statement evaluates a condition. If the condition is true, the code block within the if statement is executed.
- The else statement follows an if statement and executes its block of code if the condition in the if statement is false.

Syntax:

```
if (condition) {
  // Code to be executed if condition is true
} else {
  // Code to be executed if condition is false
}
```

Example:

```
let age = 20;

if (age >= 18) {
  console.log("You are eligible to vote.");
} else {
  console.log("You are not eligible to vote.");
}
// Output: "You are eligible to vote."
```

---

2. If-Else If-Else Ladder

Explanation:

- The if-else if-else ladder is used when you need to check multiple conditions. It allows you to test several conditions in sequence.

Syntax:

```
if (condition1) {
  // Code to be executed if condition1 is true
} else if (condition2) {
  // Code to be executed if condition2 is true
} else {
  // Code to be executed if none of the conditions are true
}
```

Example:

```
let score = 75;

if (score >= 90) {
  console.log("Grade: A");
} else if (score >= 80) {
  console.log("Grade: B");
```

```
} else if (score >= 70) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
}
// Output: "Grade: C"
```

---

3. Nested If-Else

Explanation:

- Nested if-else statements allow you to place an if-else statement inside another if-else statement. This is useful when you need to make multiple decisions based on different conditions.

Syntax:

```
if (condition1) {
  if (condition2) {
    // Code to be executed if both condition1 and condition2 are true
  } else {
    // Code to be executed if condition1 is true and condition2 is
false
  }
} else {
  // Code to be executed if condition1 is false
}
```

Example:

```
let year = 2024;
let isLeapYear;

if (year % 4 === 0) {
  if (year % 100 === 0) {
    if (year % 400 === 0) {
      isLeapYear = true;
    } else {
```

```
      isLeapYear = false;
    }
  } else {
    isLeapYear = true;
  }
} else {
  isLeapYear = false;
}

console.log(isLeapYear ? "Leap year" : "Not a leap year");
// Output: "Leap year"
```

---

4. Ternary Operator (Shorthand If-Else)

Explanation:

- The ternary operator is a shorthand way of writing an if-else statement. It takes three operands: a condition, an expression to execute if the condition is true, and an expression to execute if the condition is false.

Syntax:

```
condition ? expressionIfTrue : expressionIfFalse;
```

Example:

```
let age = 16;
let canVote = (age >= 18) ? "Yes" : "No";

console.log("Can vote: " + canVote);
// Output: "Can vote: No"
```

---

5. Switch Statement (Alternative to If-Else)

Explanation:

- The switch statement is an alternative to multiple if-else if statements. It compares the value of a variable against multiple cases and executes the corresponding block of code.

Syntax:

```
switch (expression) {
  case value1:
    // Code to be executed if expression === value1
    break;
  case value2:
    // Code to be executed if expression === value2
    break;
  default:
    // Code to be executed if expression doesn't match any case
}
```

Example:

```
let day = "Monday";

switch (day) {
  case "Monday":
    console.log("Start of the work week.");
    break;
  case "Friday":
    console.log("End of the work week.");
    break;
  case "Saturday":
  case "Sunday":
    console.log("Weekend!");
    break;
  default:
    console.log("Midweek day.");
```

```
}
// Output: "Start of the work week."
```

---

3. While Loop

Explanation:

- The while loop is used to repeatedly execute a block of code as long as a specified condition is true.
- Before each iteration, the condition is checked. If the condition is true, the code block is executed. If the condition is false, the loop stops.

Syntax:

```
while (condition) {
  // Code to be executed repeatedly as long as the condition is true
}
```

Example:

```
let count = 0;

while (count < 5) {
  console.log("Count is: " + count);
  count++;
}
// Output:
// Count is: 0
// Count is: 1
// Count is: 2
// Count is: 3
// Count is: 4
```

---

2. Do-While Loop

Explanation:

- The do-while loop is similar to the while loop, but with one key difference: the code block is executed **at least once** before the condition is tested.
- The condition is checked **after** the code block has executed, so the loop runs at least once, even if the condition is false initially.

Syntax:

```
do {
  // Code to be executed at least once and then repeatedly as long as the condition is true
} while (condition);
```

Example:

```
let count = 0;

do {
  console.log("Count is: " + count);
  count++;
} while (count < 5);
// Output:
// Count is: 0
// Count is: 1
// Count is: 2
// Count is: 3
// Count is: 4
```

---

3. Key Differences Between While and Do-While Loops

- **Condition Check Timing:**

- **While Loop:** The condition is checked before executing the loop's code block. If the condition is false initially, the code block may not execute at all.
- **Do-While Loop:** The condition is checked after executing the loop's code block. This guarantees that the code block executes at least once, even if the condition is false initially.

For Loop:

The **for** loop is one of the most commonly used loops in JavaScript. It is designed to execute a block of code a specific number of times, making it ideal for scenarios where you know in advance how many times you want to iterate.

1. Basic For Loop

Explanation:

- The for loop is used to iterate over a block of code a set number of times. It consists of three parts: initialization, condition, and increment/decrement.
- **Initialization:** Sets a starting point, usually by declaring a loop counter variable.
- **Condition:** The loop continues to run as long as this condition is true.
- **Increment/Decrement:** Updates the loop counter after each iteration.

Syntax:

```
for (initialization; condition; increment/decrement) {

  // Code to be executed on each iteration
```

```
}
```

Example:

```
for (let i = 0; i < 5; i++) {
  console.log("Iteration number: " + i);
}
// Output:
// Iteration number: 0
// Iteration number: 1
// Iteration number: 2
// Iteration number: 3
// Iteration number: 4
```

---

2. For-In Loop

Explanation:

- The for-in loop is used to iterate over the properties of an object.
- It loops through the keys (property names) of an object.

Syntax:

```
for (let key in object) {
  // Code to be executed for each key in the object
}
```

Example:

```
const person = {
  name: "suresh",
  age: 35,
  city: "chennai"
};

for (let key in person) {
  console.log(key + ": " + person[key]);
}
```

---

3. For-Of Loop

Explanation:

- The for-of loop is used to iterate over the values of an iterable object, such as arrays, strings, or NodeLists.
- It loops through the elements (values) of an iterable object.

Syntax:

```
for (let element of iterable) {
  // Code to be executed for each element in the iterable
}
```

Example:

```
const numbers = [1, 2, 3, 4, 5];
for (let num of numbers) {
```

```
  console.log("Number: " + num);
}
// Output:
// Number: 1
// Number: 2
// Number: 3
// Number: 4
// Number: 5
```

---

4. Nested For Loop

Explanation:

- A nested for loop is a loop inside another loop.
- The inner loop will be executed for each iteration of the outer loop.

Syntax:

```
for (let i = 0; i < outerLimit; i++) {
  for (let j = 0; j < innerLimit; j++) {
    // Code to be executed for each pair of i and j
  }
}
```

Example:

```
for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
```

```
    console.log("i = " + i + ", j = " + j);
  }
}
// Output:
// i = 1, j = 1
// i = 1, j = 2
// i = 1, j = 3
// i = 2, j = 1
// i = 2, j = 2
// i = 2, j = 3
// i = 3, j = 1
// i = 3, j = 2
// i = 3, j = 3
```

Day4 :

Hoisting & Scope in JavaScript

1. Hoisting:

Explanation:

- **Hoisting** is a JavaScript mechanism where variables and function declarations are moved to the top of their containing scope during the compile phase before the code is executed.
- This means that you can use variables and functions before they are declared in the code.

Example with Variable Hoisting:

```
console.log(x); // Output: undefined
var x = 5;
```

Example with Function Hoisting:

```
display(); // Output: "Hello, Team!"

function display() {
  console.log("Hello, Team!");
}
```

2. Scope:

Explanation:

- **Scope** refers to the accessibility of variables and functions in different parts of your code.
- JavaScript has a **Global Scope** and **Local Scope** (function or block scope).

Global Scope:

- Variables declared outside any function or block are in the global scope, accessible from anywhere in the code.

Local Scope:

- Variables declared inside a function are in the local scope and can only be accessed within that function.

Example:

```
var globalVar = "I am global";

function myFunction() {
  var localVar = "I am local";
  console.log(globalVar); // Accessible
  console.log(localVar);  // Accessible
}

myFunction();
console.log(localVar); // Error: localVar is not defined
```

Block Scope (with `let` and `const`):

- Variables declared with `let` or `const` inside a block (`{}`) are only accessible within that block.

Example:

```
if (true) {
  let blockVar = "I am block-scoped";
  console.log(blockVar); // Accessible
}

console.log(blockVar); // Error: blockVar is not defined
```

Arrays & Array Methods in JavaScript

1. Arrays:

Explanation:

- An **array** is a collection of items stored at contiguous memory locations.
- Arrays can store multiple values in a single variable, and each value is accessed using an index.

Example:

```
let fruits = ["apple", "banana", "mango"];
console.log(fruits[0]); // Output: "apple"
console.log(fruits.length); // Output: 3
```

2. Common Array Methods:

`push()` and `pop()`: Add or remove elements from the end of an array.

```
fruits.push("orange");
console.log(fruits); // ["apple", "banana", "mango", "orange"]

fruits.pop();
console.log(fruits); // ["apple", "banana", "mango"]
```

`shift()` and `unshift()`: Add or remove elements from the beginning of an array.

```
fruits.unshift("grape");
console.log(fruits); // ["grape", "apple", "banana", "mango"]

fruits.shift();
console.log(fruits); // ["apple", "banana", "mango"]
```

`slice()` and `splice()`: Extract or modify parts of an array.

```
let slicedFruits = fruits.slice(1, 3);
console.log(slicedFruits); // ["banana", "mango"]

fruits.splice(1, 1, "kiwi");
console.log(fruits); // ["apple", "kiwi", "mango"]
```

`map()`: Creates a new array by applying a function to each element of the array.

```
let upperCaseFruits = fruits.map(fruit => fruit.toUpperCase());
console.log(upperCaseFruits); // ["APPLE", "KIWI", "MANGO"]
```

`filter()`: Creates a new array with elements that pass a test defined in a function.
javascript
Copy code

```
let filteredFruits = fruits.filter(fruit => fruit.length > 4);
console.log(filteredFruits); // ["apple", "mango"]
```

`forEach()`: Executes a provided function once for each array element.

```
fruits.forEach(fruit => console.log(fruit));
// Output:
// "apple"
```

```
// "kiwi"
// "mango"
```

Object and JSON Iteration in JavaScript

1. Iterating Over Objects:

Using for-in Loop:

```
let person = {
  name: "Suresh",
  age: 35,
  city: "Coimbatoe"
};

for (let key in person) {
  console.log(key + ": " + person[key]);
}
// Output:
// "name: Suresh"
// "age: 35"
// "city: Coimbatoe "
```

-

Using Object.keys() and Object.values():

```
let keys = Object.keys(person);
let values = Object.values(person);

console.log(keys);   // ["name", "age", "city"]
```

```
console.log(values); // ["John", 30, "New York"]
```

2. Iterating Over JSON:

- Iterating over JSON data in JavaScript is a common task, especially when dealing with API responses or configuration data. JSON (JavaScript Object Notation) is typically represented as an object or array of objects.
- Let's explore how to iterate over JSON data using an example.

Example:

```
 let jsonData = [{
        name: "suresh",
        age: 35,
        city: "coimbatore"
      },
      {
        name: "vel",
        age: 33,
        city: "salem"
      },
      {
        name: "rajesh",
        age: 34,
        city: "erode"
      }]

jsonData.forEach((i,ind)=>{
    console.log(i.name);
    console.log(i.age);
    console.log(i.city);
})

let result = jsonData.map((i, ind) => {
    if (i.age > 33) { // Condition: Age must be greater than 25
```

```
        console.log(i.name);
        console.log(i.age);
        console.log(i.city);
        return { name: i.name, age: i.age, city: i.city }; //
Returning the filtered user object
    }
    return null; // Return null if the condition is not met
})
// .filter(item => item !== null); // Filter out null values

console.log(result);
```

Day1 : HTML

1. What is HTML & Why Use It?

HTML (HyperText Markup Language) is the standard markup language used to create web pages. It provides the structure of a webpage by using elements such as headings, paragraphs, links, images, lists, and more.

Hypertext Markup Language (HTML) is named this way because:

- **Hypertext**: It enables linking different pieces of content together through hyperlinks.
- **Markup**: It uses tags to "mark up" text and define the structure and layout of a webpage.
- **Language**: It follows a set of rules and syntax that browsers interpret to display content correctly.

Why Use HTML?

- **Structure Content**: HTML provides a structured way to define the content of a webpage, such as text, images, and multimedia.
- **Semantic Meaning**: HTML allows you to use semantic elements that convey meaning to the content (e.g., `<header>`, `<footer>`, `<article>`).
- **Accessibility**: Using HTML properly ensures that web content is accessible to all users, including those using screen readers.
- **SEO(Search Engine Optimization)**: Proper HTML structure helps search engines understand the content of your webpage, which can improve your website's visibility.

When you open a webpage in a browser, here's what happens step-by-step:

Request and Response:

- When you enter a URL in your browser's address bar or click a link, the browser sends an HTTP request to the server hosting the website.
- The server responds by sending back the HTML file (along with other resources like CSS, JavaScript, images, etc.).

Parsing the HTML Document:

- The browser receives the HTML document from the server and starts reading it from top to bottom.

- The browser parses the HTML document into a **Document Object Model (DOM)**, a tree-like structure that represents the content of the webpage.

  Building the DOM Tree:

- The browser reads each HTML tag and creates corresponding DOM nodes. For example, a `<p>` tag creates a paragraph node, and an `<img>` tag creates an image node in the DOM tree.
- The DOM tree represents the hierarchical structure of the webpage, with parent-child relationships between elements.

Simplified Workflow:

- **HTML Parsing**: The browser reads the HTML and constructs the DOM.
- **CSS Parsing**: The browser fetches and parses CSS files and applies styles.
- **JavaScript Execution**: The browser runs JavaScript code, manipulating the DOM and CSS as needed.
- **Rendering**: The browser paints the content onto the screen based on the DOM, CSS, and JavaScript.

2. Basic HTML Tags

HTML tags are used to create elements, and each tag usually comes in pairs: an opening tag and a closing tag. Here's a list of some basic HTML tags:

- **`<html>`**: The root element of an HTML document.
- **`<head>`**: Contains meta-information about the document (like the title and link to CSS files).
- **`<title>`**: Sets the title of the webpage, which appears on the browser tab.
- **`<body>`**: Contains the content of the webpage, such as text, images, and other elements.
- **`<h1>` to `<h6>`**: Header tags used to define headings. `<h1>` is the largest, and `<h6>` is the smallest.
- **`<p>`**: Defines a paragraph.
- **`<a>`**: Creates a hyperlink. The `href` attribute specifies the URL of the link.
- **`<img>`**: Embeds an image. The `src` attribute specifies the path to the image file, and `alt` provides alternative text for accessibility.
- **`<ul>`** and **`<li>`**: Used for unordered (bulleted) lists.

- **`<ol>`** and **`<li>`**: Used for ordered (numbered) lists.

Key Differences Between Tags and Attributes

Feature	Tags	Attributes
Definition	Structural elements that define content.	Properties that modify or describe elements.
Placement	Placed inside angle brackets, defining an element.	Placed inside the opening tag of an element.
Syntax	`<tagname>...</tagname>` or `<tagname />`	`attribute="value"`
Purpose	To structure and format the document.	To provide additional information or modify behavior.
Examples	`<p>`, `<h1>`, `<div>`, `<img>`, `<a>`	`href`, `src`, `alt`, `class`, `id`

Example:

```
<!DOCTYPE html>
<html>
<head>
    <title>Basic HTML Example</title>
</head>
<body>
```

```
    <h1>Welcome to My Website</h1>
    <p>This is a paragraph of text on my website.</p>
    <a href="https://www.example.com">Visit Example.com</a>
    <img src="image.jpg" alt="A descriptive image">
    <ul>
        <li>Item 1</li>
        <li>Item 2</li>
    </ul>
</body>
</html>
```

3. `<div>` vs `<span>`

- **`<div>`**: Used to group block-level elements (e.g., <p>, <h1>) to apply CSS styles or JavaScript. It creates a block-level element, which means it starts on a new line and takes up the full width available.
- **`<span>`**: Used to group inline elements (e.g., parts of text within a paragraph) to apply CSS styles or JavaScript. It is an inline element, which means it does not start on a new line and only takes up as much width as necessary.

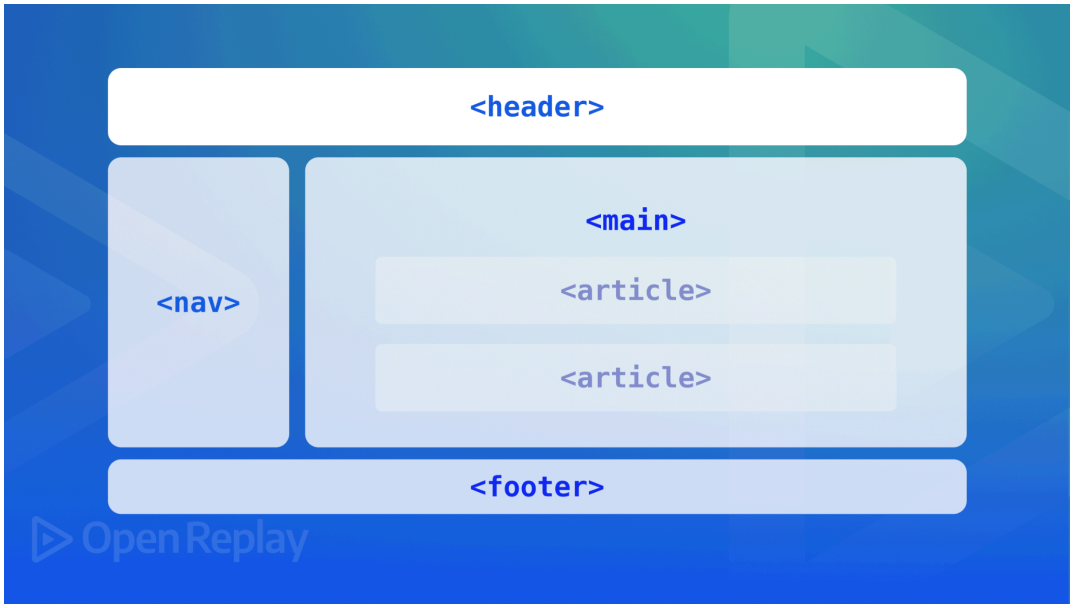
Example:

```
<div style="border: 1px solid black; padding: 10px;">
    This is a <div> element. It is used to group block-level elements.
</div>

<p>This is a paragraph with a <span style="color: red;">highlighted
text</span> inside it.</p>
```

4. Semantic Tags

Semantic HTML tags provide meaning to the web content, improving accessibility and SEO. These tags clearly define the purpose of the elements.



Common Semantic Tags:

- **`<header>`**: Defines the header section of a webpage or a section, usually containing navigation links, a logo, etc.
- **`<nav>`**: Defines a navigation block, containing links to other pages or sections within the same page.
- **`<main>`**: Represents the main content of the body of a document.
- **`<article>`**: Represents a self-contained piece of content that could be distributed or reused independently.
- **`<section>`**: Defines a section in a document, such as chapters, headers, or any thematic grouping of content.
- **`<aside>`**: Represents content indirectly related to the main content, like sidebars or advertisements.
- **`<footer>`**: Represents the footer section of a webpage or a section, often containing the author's information, links to related documents, etc.

Example:

```
<header>
    <h1>Website Header</h1>
    <nav>
        <a href="#home">Home</a>
        <a href="#about">About</a>
```

```
    </nav>
</header>

<main>
    <article>
        <h2>Article Title</h2>
        <p>This is an article paragraph.</p>
    </article>
</main>

<aside>
    <p>Related links or advertisements go here.</p>
</aside>

<footer>
    <p>Website footer information here.</p>
</footer>
```

5. Forms

Forms are used to collect user input. A form typically contains input elements like text fields, checkboxes, radio buttons, and submit buttons.

Basic Form Elements:

- **`<form>`**: The container for all form elements.
- **`<input>`**: Defines an input field where the user can enter data. The `type` attribute specifies the type of input (e.g., text, password, email).
- **`<textarea>`**: A multi-line text input field.
- **`<button>`**: Defines a clickable button, often used to submit the form.
- **`<label>`**: Defines a label for a form element and improves accessibility.

Example:

```
<form action="/submit-form" method="post">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>

    <label for="email">Email:</label>
```

```
    <input type="email" id="email" name="email" required>

    <button type="submit">Submit</button>
</form>
```

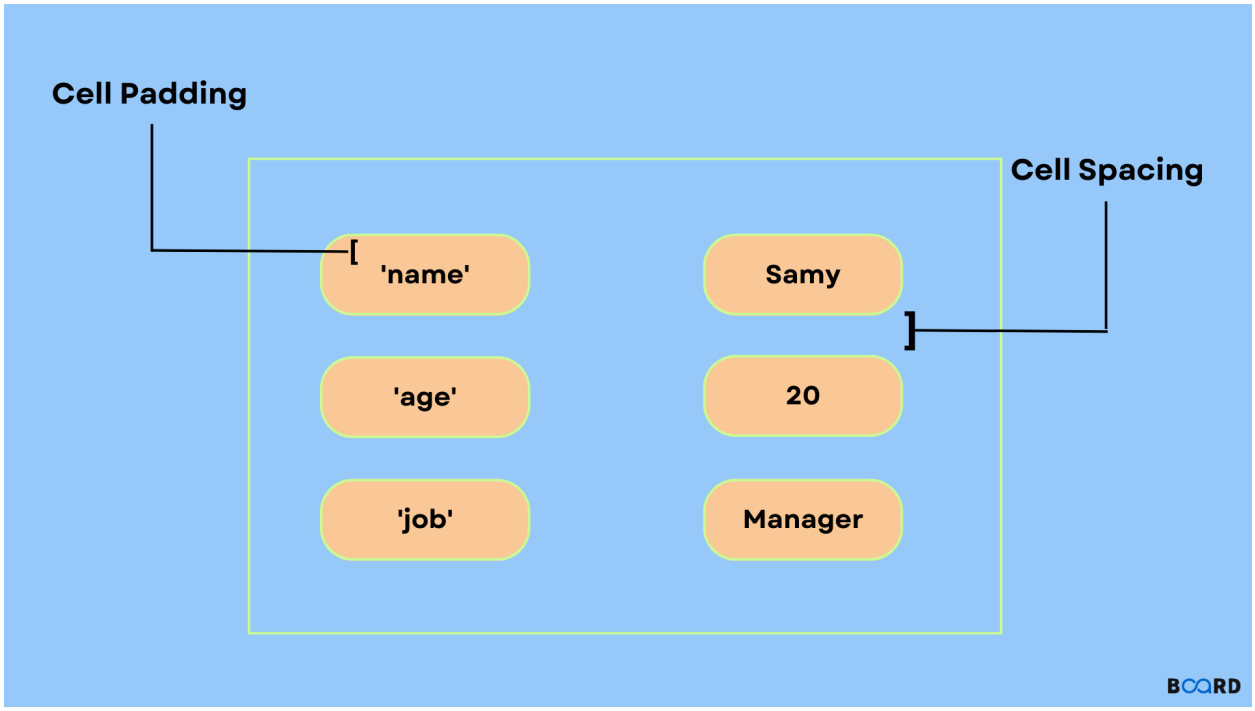
6. Tables

Tables are used to display data in a structured grid format, making it easier to read and understand.

Basic Table Elements:

- **`<table>`**: The container for all table-related elements.
- **`<tr>`**: Table row.
- **`<th>`**: Table header, typically used at the top of a column.
- **`<td>`**: Table data cell.

Attributes like : cellspadding , cellspacing width, height, border, colspan, rowspan



Example:

```
<table border="1">
    <tr>
        <th>Name</th>
        <th>Age</th>
        <th>City</th>
    </tr>
    <tr>
        <td>John</td>
        <td>25</td>
        <td>New York</td>
    </tr>
    <tr>
        <td>Jane</td>
        <td>30</td>
        <td>Los Angeles</td>
    </tr>
</table>
```

Day6:

1. What is CSS?

CSS (Cascading Style Sheets) is a stylesheet language used to describe the presentation of a document written in HTML or XML. CSS defines **how HTML elements should be displayed on screen,** paper, or in other media. **It helps to control the layout, colors, fonts, and overall visual appearance of web pages.**

Why use CSS?

- **Separation of Concerns**: CSS allows you to separate content (HTML) from presentation (styles), making it easier to maintain and update.
- **Consistency**: You can apply consistent styling across multiple pages using a single CSS file.

- **Efficiency**: CSS reduces code repetition and allows you to style multiple elements with a single rule.
- **Flexibility**: It offers a variety of styling options for layout, typography, and visual effects.

2. CSS Selectors: Class vs. ID

CSS selectors are **patterns used to select the elements you want to style.** Two of the most commonly used selectors are **class** and **ID**.

Class Selector (`.`)

- **Definition**: A class selector selects elements with a specific class attribute. You can apply a class to **multiple elements on a page.**
- **Syntax**: The class selector is defined using a period (`.`) followed by the class name.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <style>
        /* CSS class selector */
        .highlight {
            color: blue;
            font-weight: bold;
        }
    </style>
    <title>Class Selector Example</title>
</head>
<body>
    <p class="highlight">This text is highlighted using a class
selector.</p>
    <p class="highlight">Another highlighted paragraph using the same
class.</p>
    <p>This paragraph is not highlighted.</p>
</body>
</html>
```

In this example, the `.highlight` class is applied to two paragraphs, changing their text color to blue and making them bold.

ID Selector (#)

- **Definition**: An ID selector **selects an element** with a specific ID attribute. The ID must be **unique within a page**, meaning it can only be used once.
- **Syntax**: The ID selector is defined using a hash (#) followed by the ID name.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <style>
        /* CSS ID selector */
        #uniqueElement {
            background-color: yellow;
            padding: 10px;
        }
    </style>
    <title>ID Selector Example</title>
</head>
<body>
    <p id="uniqueElement">This paragraph has a unique background color
using an ID selector.</p>
    <p>This paragraph is not affected by the ID selector.</p>
</body>
</html>
```

In this example, the `#uniqueElement` ID is applied to one paragraph, giving it a yellow background and padding.

Key Differences Between Class and ID Selectors:

- **Class Selector (`.`)**:
    - Can be applied to multiple elements.

    - Great for styling multiple elements with similar styles.
- **ID Selector (#)**:
    - Must be unique on a page (used only once).
    - Best for styling a specific, unique element.

3. Display Property: `inline`, `block`, and `inline-block`

The **display** property in CSS determines how an **element is displayed on a webpage.**

`inline`

- **Definition**: `inline` elements **do not start on a new line and only take up as much width** as necessary. They do not force a line break after the element.
- **Common Uses**: Formatting text within a paragraph, such as **`<span>`**, `<a>`, `<strong>`, and `<em>`.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <style>
        .inline-example {
            display: inline;
            color: red;
        }
    </style>
    <title>Inline Display Example</title>
</head>
<body>
    <p>This is an <span class="inline-example">inline element</span>
within a paragraph.</p>
</body>
</html>
```

In this example, the `<span>` with the class `.inline-example` is displayed inline, without breaking the paragraph's flow.

`block`

- **Definition**: `block` elements **start on a new line and take up the full width** available. They create a block-level box.
- **Common Uses**: Structural elements like `<div>`, `<h1>` to `<h6>`, `<p>`, `<ul>`, and `<ol>`.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <style>
        .block-example {
            display: block;
            background-color: lightgrey;
            margin-bottom: 10px;
        }
    </style>
    <title>Block Display Example</title>
</head>
<body>
    <div class="block-example">This is a block-level element.</div>
    <div class="block-example">Another block-level element.</div>
</body>
</html>
```

In this example, each `<div>` with the class `.block-example` is displayed as a block, starting on a new line and taking up the full width.

`inline-block`

- **Definition**: `inline-block` elements are like `inline` elements but **allow you to set a width and height.** They do not force a line break after the element, and they respect vertical alignment.
- **Common Uses**: Elements that **need to be inline but also need block-level styling,** such as buttons or images.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <style>
        .inline-block-example {
            display: inline-block;
            width: 100px;
            height: 100px;
            background-color: coral;
            margin-right: 10px;
        }
    </style>
    <title>Inline-Block Display Example</title>
</head>
<body>
    <div class="inline-block-example">Box 1</div>
    <div class="inline-block-example">Box 2</div>
</body>
</html>
```

In this example, the two `<div>` elements are displayed as inline blocks, sitting next to each other without breaking to a new line.

4. Intro to Pseudo-Classes: `link`, `visited`, `hover`, `active`

Pseudo-classes are **keywords added to selectors** that specify a special state of the selected elements. They allow you to style elements based on their state, such as **when a link is hovered over or when an input field is focused.**

Common Pseudo-Classes

1. **`:link`**: Targets all unvisited links.
2. **`:visited`**: Targets all visited links.
3. **`:hover`**: Targets an element when the user hovers over it with a mouse.

4. **`:active`**: Targets an element when it is in the process of being activated (e.g., when a mouse button is clicked down on a link).

Example of Pseudo-Classes:

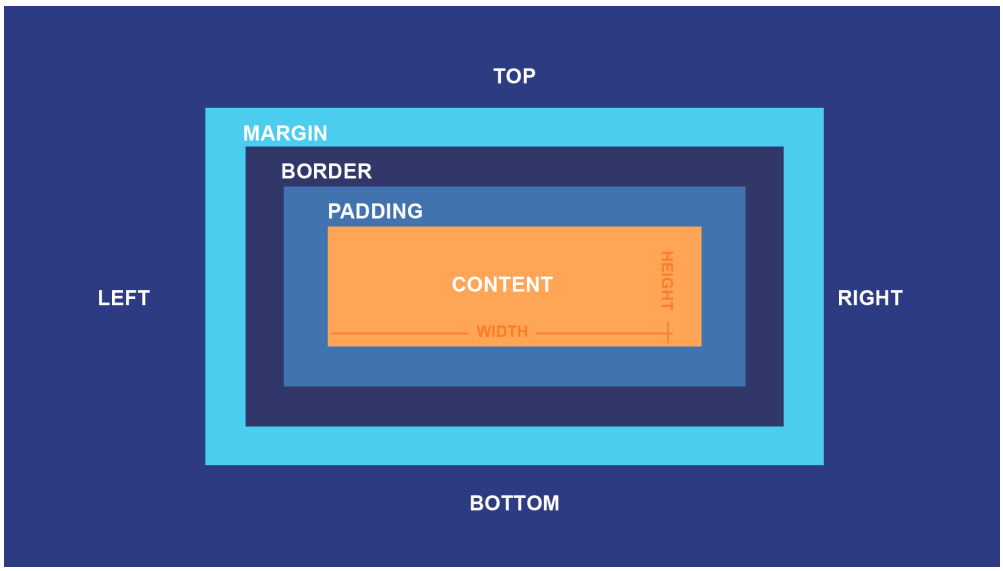
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <style>
        /* Unvisited link */
        a:link {
            color: blue;
        }
        /* Visited link */
        a:visited {
            color: purple;
        }
        /* Hovered link */
        a:hover {
            color: red;
        }
        /* Active link */
        a:active {
            color: green;
        }
    </style>
    <title>Pseudo-Classes Example</title>
</head>
<body>
    <p>Here are some links to demonstrate pseudo-classes:</p>
    <a href="https://example.com">Visit Example (Unvisited
Link)</a><br>
    <a href="https://example.com">Visit Example Again (Visited
Link)</a>
</body>
</html>
```

Day7:

1. What is the Box Model?

The **CSS Box Model** is a fundamental concept that describes **how elements are structured and rendered on a webpage.** Every HTML element can be considered a box, and the box model consists of **several layers**: **content, padding, border,** and **margin**.

- **Content**: The innermost part of the box where text and images appear.
- **Padding**: Space between the content and the border. The padding adds space inside the element but is still considered part of the element's background.
- **Border**: A line surrounding the padding (if any) and content.
- **Margin**: Space outside the border that separates the element from its neighbors.



Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>CSS Box Model Example</title>
    <style>
```

```
        .box {
            width: 200px;
            height: 100px;
            padding: 20px;
            border: 5px solid #333;
            margin: 10px;
            background-color: lightblue;
        }
    </style>
</head>
<body>
    <div class="box">This is a box model example.</div>
</body>
</html>
```

2. Padding and Margin

- **Padding**: Adds space **inside** the element, between the content and the border.
- **Margin**: Adds space **outside** the element, separating it from other elements.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Padding and Margin Example</title>
    <style>
        .container {
            width: 300px;
            background-color: #f0f0f0;
        }
        .padding-box {
            padding: 20px;
            background-color: lightgreen;
        }
```

```
        .margin-box {
            margin: 20px;
            background-color: lightcoral;
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="padding-box">This box has padding.</div>
        <div class="margin-box">This box has margin.</div>
    </div>
</body>
</html>
```

Different Ways to Set Padding and Margin Values

1. **Single Value**:
    - Applies the same value to all four sides (top, right, bottom, and left).
    - **Syntax**: padding: value; or margin: value;

Example:

```
.box {
    padding: 20px;  /* Adds 20px padding to all sides */
    margin: 10px;   /* Adds 10px margin to all sides */
}
```

2. **Two Values**:
    - The first value applies to the top and bottom, and the second value applies to the left and right.
    - **Syntax**: padding: vertical horizontal; or margin: vertical horizontal;

Example:

```
.box {
    padding: 10px 20px;  /* 10px top and bottom, 20px left and right */
    margin: 5px 15px;    /* 5px top and bottom, 15px left and right */
}
```

3. **Three Values**:
   - The first value applies to the top, the second value applies to the left and right, and the third value applies to the bottom.
   - **Syntax**: padding: top horizontal bottom; or margin: top horizontal bottom;

Example:

```
.box {
    padding: 10px 20px 30px;  /* 10px top, 20px left and right, 30px bottom */
    margin: 5px 15px 10px;    /* 5px top, 15px left and right, 10px bottom */
}
```

4. **Four Values**:
   - Applies different values to each side in the order: top, right, bottom, left.
   - **Syntax**: padding: top right bottom left; or margin: top right bottom left;

Example:

```
.box {
    padding: 5px 10px 15px 20px;  /* 5px top, 10px right, 15px bottom, 20px left */
    margin: 10px 5px 0 15px;      /* 10px top, 5px right, 0px bottom, 15px left */
}
```

5. **Shorthand for Each Side**:
    - You can set padding and margin for individual sides using specific properties.
    - **Syntax**:
        - padding-top: value;
        - padding-right: value;
        - padding-bottom: value;
        - padding-left: value;
        - margin-top: value;
        - margin-right: value;
        - margin-bottom: value;
        - margin-left: value;

Example:

```
.box {
    padding-top: 10px;
    padding-right: 15px;
    padding-bottom: 20px;
    padding-left: 25px;

    margin-top: 5px;
    margin-right: 10px;
    margin-bottom: 15px;
    margin-left: 20px;
}
```

6. **Using Percentage (%)**:
    - You can specify padding and margin values in percentages. These percentages are relative to the width of the containing element.
    - **Syntax**: padding: %; or margin: %;

Example:

```
.box {
    padding: 5%;   /* Padding is 5% of the width of the
containing element */
```

```
    margin: 10%;   /* Margin is 10% of the width of the
containing element */

}

We can give multiple values(10% 20% 10% 20%) as like px.
```

3. Box Model Real-Time Visualization Using Chrome Inspect Tools

To see the Box Model in action:

1. **Open Chrome Developer Tools**: Right-click on an element on a webpage and select **Inspect**.
2. **View the Box Model**: In the **Elements** tab, select an element and scroll down to the **Styles** panel. You will see a visual representation of the box model (content, padding, border, margin).
3. **Adjust the Values**: You can change the values for padding, border, and margin in real-time and see how the element adjusts on the page.

4. CSS Positions: Relative, Absolute, Static, Fixed, sticky

CSS positioning allows you to define how elements are placed on a page.

Static: The default positioning for all elements. The element is placed according to the normal flow of the document.

```
.static-box {
    position: static;
}
```

Relative: An element with `position: relative;` is positioned relative to its normal position in the document flow. It remains in the normal flow, **but you can adjust its position using `top`, `right`, `bottom`, or `left` properties**. This means that even after shifting, the space it would normally occupy is still reserved in the document flow.

```
.box2 {
width: 100px;
height: 100px;
```

```
background-color: lightcoral;
position: relative;
left: 20px; /* Moves the element 20px to the right from its original
position */
top: 10px; /* Moves the element 10px down from its original position
*/ }
```

Explanation:

> element is shifted 20 pixels to the right and 10 pixels down from its original position. The space it would normally occupy is still preserved, meaning the **content around it does not shift like padding.**

Absolute: Positioned relative to the nearest positioned ancestor (relative, absolute, or fixed). If none exists, it is positioned relative to the `<body>`.

```
.absolute-box {
    position: absolute;
    top: 50px;
    left: 30px;
}
```

Fixed: Positioned relative to the browser window. The element stays in the same position even when scrolling the page.

```
.fixed-box {
    position: fixed;
    bottom: 0;
    right: 0;
}
```

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>CSS Positions Example</title>
    <style>
        .static-box {
            position: static;
            width: 100px;
            height: 100px;
            background-color: lightgray;
        }

        .relative-box {
            position: relative;
            left: 20px;
            width: 100px;
            height: 100px;
            background-color: lightblue;
        }

        .absolute-box {
            position: absolute;
            top: 50px;
            left: 50px;
            width: 100px;
            height: 100px;
            background-color: lightgreen;
        }

        .fixed-box {
            position: fixed;
            bottom: 0;
            right: 0;
            width: 100px;
            height: 100px;
            background-color: lightcoral;
        }
    </style>
</head>
```

```
<body>
    <div class="static-box">Static</div>
    <div class="relative-box">Relative</div>
    <div class="absolute-box">Absolute</div>
    <div class="fixed-box">Fixed</div>
</body>
</html>
```

5. Colors: Solid & Linear Gradients

Solid Colors: Plain, single-color backgrounds. Defined using keywords (e.g., `red`), HEX codes (`#FF0000`), RGB (`rgb(255, 0, 0)`), or HSL (`hsl(0, 100%, 50%)`).

Color picker : https://htmlcolorcodes.com/

1. HEX Codes

Definition:

- **HEX** stands for **Hexadecimal**, a base-16 number system used in CSS to represent colors. It consists of a hash (#) followed by six hexadecimal digits.

Structure:

- HEX codes have the format #RRGGBB, where:
    - **RR** represents the red component (00 to FF).
    - **GG** represents the green component (00 to FF).
    - **BB** represents the blue component (00 to FF).

Examples:

- **#FF0000** - Full red with no green or blue.
- **#00FF00** - Full green with no red or blue.
- **#0000FF** - Full blue with no red or green.
- **#FFFFFF** - White (maximum red, green, and blue).
- **#000000** - Black (no red, green, or blue).

```
body {

   background-color: #ff5733; /* A shade of orange */

}
```

2. RGB (Red, Green, Blue)

Definition:

- **RGB** stands for **Red, Green, Blue**, which are the primary colors of light. The RGB model is based on combining these three colors to create a broad spectrum of colors.

Structure:

- RGB values are specified with the `rgb()` function in CSS, with the format `rgb(red, green, blue)`.
- Each component (**red**, **green**, **blue**) takes a value from **0 to 255**.

Examples:

- `rgb(255, 0, 0)` - Red.
- `rgb(0, 255, 0)` - Green.
- `rgb(0, 0, 255)` - Blue.
- `rgb(255, 255, 255)` - White.
- `rgb(0, 0, 0)` - Black.

```
p {
  color: rgb(75, 0, 130); /* Indigo color */
}
```

3. HSL (Hue, Saturation, Lightness)

Definition:

- **HSL** stands for **Hue, Saturation, Lightness**. This model represents colors in a way that is closer to human perception.

Structure:

- HSL values are specified with the `hsl()` function in CSS, with the format `hsl(hue, saturation, lightness)`.
    - **Hue**: Represents the type of color (0 to 360 degrees on the color wheel, where 0 is red, 120 is green, and 240 is blue).
    - **Saturation**: Represents the intensity of the color (0% to 100%, where 0% is a shade of gray and 100% is full color).

- **Lightness**: Represents the brightness of the color (0% to 100%, where 0% is black, 50% is normal, and 100% is white).

Examples:

- `hsl(0, 100%, 50%)` - Red.
- `hsl(120, 100%, 50%)` - Green.
- `hsl(240, 100%, 50%)` - Blue.
- `hsl(0, 0%, 100%)` - White.
- `hsl(0, 0%, 0%)` - Black.

```
div {

   background-color: hsl(200, 70%, 50%); /* A shade of blue */

}
```

Ex:

```
.solid-box {
    background-color: #FF5733;
}
```

Linear Gradients: A gradual blend between two or more colors. You can specify the direction and colors.

```
.gradient-box {
    background: linear-gradient(to right, #ff7e5f, #feb47b);
}
```

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

```
    <title>CSS Colors Example</title>
    <style>
        .solid-box {
            width: 200px;
            height: 100px;
            background-color: #FF5733;
            color: white;
            text-align: center;
            line-height: 100px;
        }

        .gradient-box {
            width: 200px;
            height: 100px;
            background: linear-gradient(to right, #ff7e5f, #feb47b);
            color: white;
            text-align: center;
            line-height: 100px;
        }
    </style>
</head>
<body>
    <div class="solid-box">Solid Color</div>
    <div class="gradient-box">Gradient Color</div>
</body>
</html>
```

Day8:

CSS Flexbox: Creating Layouts and Using Media Queries

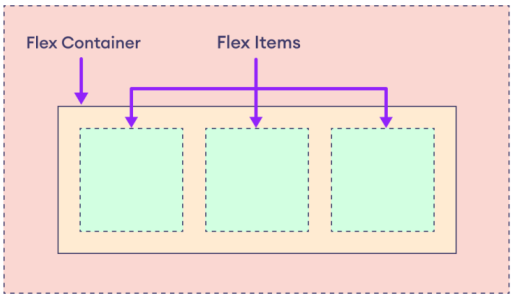
What is Flexbox?

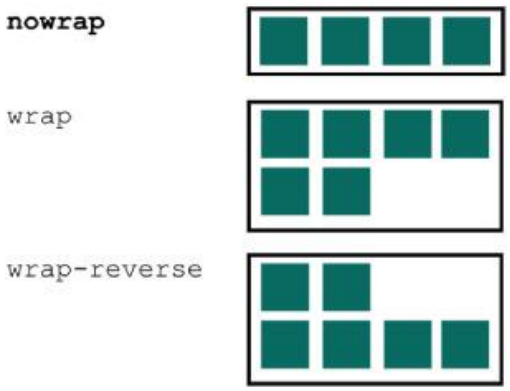
Flexbox (Flexible Box Layout) is a CSS module designed to help create layouts that are flexible and responsive. Flexbox simplifies the process of aligning, spacing, and distributing items in a

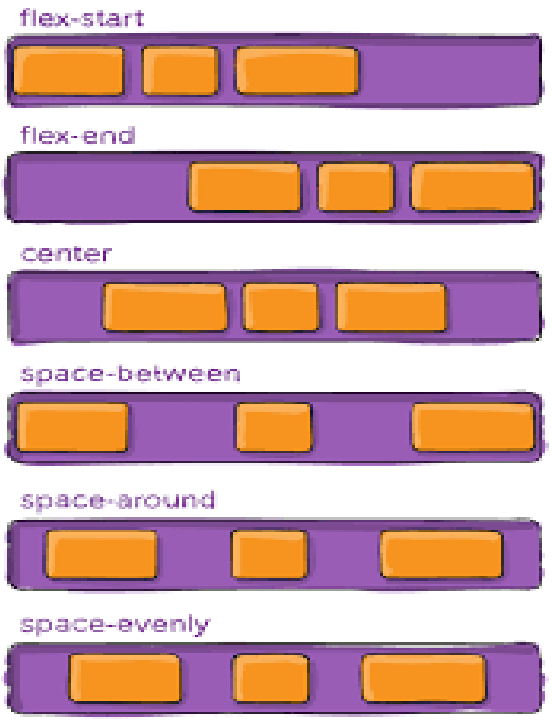
container, even when their size is unknown or dynamic. Flexbox is particularly useful for creating complex layouts and making them adapt to different screen sizes and orientations.

Key Concepts of Flexbox

1. **Flex Container**: The parent element that contains flex items. It is set to display flex using `display: flex;`.
2. **Flex Items**: The child elements within the flex container. They automatically adjust to fit the container.
3. **Main Axis**: The primary axis along which flex items are placed. It can be horizontal or vertical, depending on the `flex-direction`.
4. **Cross Axis**: The axis perpendicular to the main axis.
5. **Flex Properties**:
    - **`flex-direction`**: Determines the direction of the main axis (`row`, `column`, `row-reverse`, `column-reverse`).
    - **`justify-content`**: Aligns flex items along the main axis (start, end, center, space-between, space-around, space-evenly).
    - **`align-items`**: Aligns flex items along the cross axis (start, end, center, baseline, stretch).
    - **`align-content`**: Aligns flex lines in multi-line flex containers (start, end, center, space-between, space-around, stretch).







1. `space-between`

- **Description**: Distributes items evenly with the first item at the start and the last item at the end of the container. There is equal space between each item, but no space at the ends.

```
| Item |    space    | Item |    space    | Item |
```

- **Use Case**: This value is great when you want the items to be spread out evenly, but with no extra space at the start or end of the container.

2. `space-around`

- **Description**: Distributes items evenly, but the spaces on the ends are half the size of the space between items. This results in equal space around all items, including half spaces at the ends of the container.

```
space/2 | Item |    space    | Item |    space    | Item | space/2
```

- **Use Case**: Use `space-around` when you want some padding on the ends of the container while also keeping even spacing between items.

3. `space-evenly`

- **Description**: Distributes items so that there is equal space between all items, including at the ends of the container. All spaces (between items and on the ends) are exactly the same size.
-
  `space | Item |   space   | Item |   space   | Item | space`

Example of Flexbox Layout

Here's a basic example of using Flexbox to create a layout with a header, content area, and footer.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Flexbox Layout Example</title>
    <style>
        .container {
            display: flex;
            flex-direction: column; /* Stacks items vertically */
            height: 100vh; /* Full viewport height */
        }

        .header, .footer {
            background-color: lightblue;
            padding: 20px;
            text-align: center;
        }

        .content {
```

```
            background-color: lightgreen;
            flex: 1; /* Fills the remaining space */
            padding: 20px;
            text-align: center;
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="header">Header</div>
        <div class="content">Content Area</div>
        <div class="footer">Footer</div>
    </div>
</body>
</html>
```

Explanation of the Flexbox Example

- **`.container`**: The parent element with `display: flex` and `flex-direction: column` makes the flex items (header, content, footer) stack vertically.
- **`.content`**: The `flex: 1` property makes this element expand to fill the available space, creating a flexible layout.

Creating Responsive Layouts with Media Queries

Media Queries in CSS allow you to apply different styles based on the characteristics of the user's device, such as its screen size, orientation, and resolution. This makes your web pages responsive and adaptive to various devices.

Key Concepts of Media Queries

1. **Breakpoints**: Specific screen sizes at which the layout changes. Common breakpoints include 576px, 768px, 992px, and 1200px for mobile, tablet, laptop, and desktop, respectively.
2. **Syntax**: Media queries are written using the `@media` rule followed by conditions like `min-width` or `max-width`.

Example of Using Media Queries

Here’s an example that changes the layout of the previous Flexbox example when viewed on smaller screens.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Responsive Flexbox Layout</title>
    <style>
        .container {
            display: flex;
            flex-direction: column;
            height: 100vh;
        }

        .header, .footer {
            background-color: lightblue;
            padding: 20px;
            text-align: center;
        }

        .content {
            background-color: lightgreen;
            flex: 1;
            padding: 20px;
            text-align: center;
        }

        /* Media query for screens wider than 768px */
        @media (min-width: 768px) {
            .container {
                flex-direction: row; /* Changes direction to
horizontal */
            }


            .header, .footer {
                flex: 1; /* Header and footer share equal space */
                height: auto; /* Height adapts to content */
```

```
            }
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="header">Header</div>
        <div class="content">Content Area</div>
        <div class="footer">Footer</div>
    </div>
</body>
</html>
```

Explanation of the Media Query Example

- **Media Query**: `@media (min-width: 768px)` applies styles for devices with a width of at least 768px.
- **`.container`**: Changes the layout from vertical (`flex-direction: column`) to horizontal (`flex-direction: row`).
- **`.content`**: The `order: 2` property changes the order of the content area when viewed on larger screens.